Comprehensive Notes: Important Pandas Series Methods

Course: Data Science Mentorship Program (DSMP)

Topic: Supplementary Session on Pandas Series

1. Introduction & Setup

This session covers 10 essential Pandas Series methods that are crucial for data manipulation.

Initial Setup Code

Before starting with the methods, we need to import the necessary libraries and datasets (Virat Kohli's IPL scores, YouTube subscribers, and Bollywood movies).

```
import pandas as pd
import numpy as np

# Importing Data
subs = pd.read_csv('subscribers.csv')
vk = pd.read_csv('virat_kohli.csv', index_col='match_no', squeeze=True)
movies = pd.read_csv('bollywood.csv', index_col='movie', squeeze=True)
```

Line-by-line Explanation:

1. import pandas as pd: Importing the Pandas library for data manipulation.
2. import numpy as np: Importing the Numerical Python library.
3. pd.read_csv(...): Loading the CSV files into Pandas objects.
4. index_col: Setting a specific column as the index of the series.
5. squeeze=True: Converting a single-column DataFrame into a Series.

2. The astype() Method

The astype() method is used to change the data type of a Series.

Explanation

Sometimes data is imported in a high-memory format (like int64). If we know the range of our data is small, we can convert it to a smaller format (like int16) to save memory (reduce the memory footprint).

Code Snippet

```
# Checking current data type and size
print(vk.dtype)
print(vk.size)

# Changing type to save memory
vk_new = vk.astype('int16')
```

Line-by-line Explanation:

1. vk.dtype: Checks the current data type (usually int64).
2. vk.size: Shows the total number of items in the series.
3. vk.astype('int16'): Converts the data type from 64-bit integer to 16-bit integer.

Key Point: When dealing with large datasets, always try to use astype() to reduce memory usage.

3. The between() Method

This method checks if values in a Series fall within a specific range.

Explanation

It returns a **Boolean Series** (True/False). It is very helpful for filtering data based on a range.

Code Snippet

```
# Finding matches where Virat scored between 51 and 99
mask = vk.between(51, 99)
print(vk[mask])
```

Line-by-line Explanation:

1. vk.between(51, 99): Checks every value in the vk series. If the value is ≥ 51 and ≤ 99, it returns True, otherwise False.
2. vk[mask]: Uses the Boolean series (mask) to filter and show only the actual scores.

Important Note: Both the start and end values (51 and 99) are **inclusive**.

4. The clip() Method

The clip() method is used to "trim" values at specific thresholds.

Explanation

If a value is below the lower limit, it becomes the lower limit. If it is above the upper limit, it becomes the upper limit. Values in between remain unchanged.

Code Snippet

```
# Clipping values between 100 and 200
clipped_subs = subs.clip(100, 200)
```

Line-by-line Explanation:

1. subs.clip(100, 200): Every value in subs less than 100 becomes 100. Every value greater than 200 becomes 200.

Why use it? Useful for handling outliers or keeping data within a specific logical boundary.

5. The drop_duplicates() & duplicated() Methods

These methods help manage repetitive data.

Explanation

- drop_duplicates(): Removes duplicate values.
- duplicated(): Tells you which values are duplicates (returns Boolean).

Code Snippet

```
# Removing duplicates
temp = pd.Series([1,1,2,2,3,3,4,4])
print(temp.drop_duplicates(keep='last'))

# Checking count of duplicates in Bollywood actors
print(movies.duplicated().sum())
```

Line-by-line Explanation:

1. keep='last': When dropping duplicates, it keeps the last occurrence and deletes the previous ones. (Default is keep='first').
2. movies.duplicated(): Returns True for every value that has appeared before in the series.
3. .sum(): Adding up the True values gives the total count of duplicate entries.

6. Handling Missing Values (isnull, dropna, fillna)

Data in the real world often has "NaN" (Not a Number) values.

A. Difference between size and count()

- size: Includes everything (values + missing NaNs).
- count(): Only counts non-missing (non-null) values.

B. isnull()

Returns True if a value is missing.

```
# Count missing values
print(vk.isnull().sum())
```

C. dropna()

Removes all rows containing missing values.

```
temp_clean = temp.dropna()
```

D. fillna()

Replaces missing values with a specific value (e.g., 0 or the Mean).

```
# Filling with zero
temp_filled = temp.fillna(0)

# Filling with Mean
temp_mean_filled = temp.fillna(temp.mean())
```

7. The isin() Method

Used to check multiple conditions at once.

Explanation

Instead of writing multiple OR conditions (e.g., score == 49 or score == 99), we can pass a list of values to isin().

Code Snippet

```
# Finding matches where score was exactly 49, 99, 69, or 79
mask = vk.isin([49, 99, 69, 79])
print(vk[mask])
```

8. The apply() Method

The most powerful method for applying custom logic.

Example 1: String Manipulation

Requirement: Extract the first name of every actor in uppercase.

```
# Using Lambda for custom logic
movies.apply(lambda x: x.split()[0].upper())
```

Explanation:

1. x.split(): Splits the full name into a list (e.g., ["Akshay", "Kumar"]).
2. [0]: Takes the first item (First Name).
3. .upper(): Converts it to capital letters.

Example 2: Conditional Logic

Requirement: Label days as "Good Day" if subscribers gained > average, else "Bad Day".

```
avg = subs.mean()
subs.apply(lambda x: "Good Day" if x > avg else "Bad Day")
```

9. The copy() Method (Copy vs View)

Understanding how memory works in Pandas.

Explanation

- **View:** When you use head() or tail(), Pandas gives you a "View" of the original data. If you change the View, the **original data also changes**.
- **Copy:** Creates a completely separate replica. Changes to the copy **do not** affect the original.

Code Snippet (The Problem - View)

```
new = vk.head()
new[1] = 1 # This changes BOTH 'new' and the original 'vk'!
```

Code Snippet (The Solution - Copy)

```
new = vk.head().copy()
new[1] = 1 # Only 'new' changes; 'vk' remains safe.
```

Teacher's Tip: Always use .copy() when you want to experiment with a subset of data without risking the original dataset.

Note: These methods are fundamental building blocks. Many more exist, which are covered during the study of DataFrames.